Object Oriented Programming (CSEG1044)
Unit 06 – Lecture 02 Notes
Requirements → Use Cases → Milestones

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 06) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 Step 1: Convert scope into requirements . . . 2
5.1.1 How to write a good requirement . . . 3
5.2 Step 2: Identify actors and use cases (goals) . . . 3
5.3 Step 3: Create a use case diagram (quick guide) . . . 3
5.4 Step 4: Write the use cases (template) . . . 4
5.5 Worked example: Use case "Create Booking" (full) . . . 4
5.6 Step 5: Convert use cases into backlog items . . . 5
5.7 Step 6: Milestones (turn backlog into a plan) . . . 5

6 Deliverable-1: Use case diagram + 3 written use cases **6**

7 In-Class Activity (evidence) **6**

8 In-Class Exercises (with brief solutions) **6**

9 Self-Study Practice (no solutions) **7**

10 Checkpoint Questions (with sample answers) **7**

1 Lecture Overview

In Deliverable-0 you selected a theme and defined a scope. Now we convert that scope into **clear requirements** that can be implemented and tested. The workflow is:

Requirements → Use Cases → Milestones/Backlog

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

2 Syllabus Alignment (Unit 06)

Unit 06: requirements + use cases + project planning

3 Learning Outcomes

After this lecture, you should be able to:

- Identify actors and write use cases with success/failure scenarios and acceptance criteria.
- Create Deliverable-1: use case diagram + 3 written use cases.
- Convert requirements into milestones and update your GitHub backlog.

4 Key Terms (Quick)

- **Requirement**: what the system must do (functional) or must be (non-functional).
- **Functional requirement**: a feature/behavior (e.g., create a booking).
- **Non-functional requirement**: quality/constraint (e.g., usability, security, performance).
- **Actor**: external role interacting with the system (admin, user).
- **Use case**: goal-oriented interaction between an actor and the system.
- **Scenario**: step-by-step story of using the system (success/failure paths).
- **Acceptance criteria**: conditions to consider a use case complete (testable).
- **Definition of done (DoD)**: your team's shared checklist for when work is truly "complete".
- **Milestone**: group of backlog items delivered by a date/session.

5 Detailed Notes

5.1 Step 1: Convert scope into requirements

Your Deliverable-0 scope (Must/Should/Could/Non-goals) is still high level. Now we convert it into requirements that can be **implemented** and **tested**.

5.1.1 How to write a good requirement

Good requirements are:

- **clear:** avoid vague words like "fast" or "good",
- **specific:** mention the feature and the rule,
- **testable:** you can write 2–3 test cases from it.

Weak (avoid):

- "The system should be fast."
- "The UI should be user friendly."

Better (testable):

- "Booking cannot overlap an existing booking for the same resource."
- "Capacity must be a positive integer; otherwise show an error message and do not save."
- "On success, the record is saved in DB and appears in the list view without restarting the app."

5.2 Step 2: Identify actors and use cases (goals)

Start from your requirements and identify:

- actors (roles): Admin, User, Staff, etc.,
- goals: what each actor wants to achieve.

Each goal becomes a use case.

Example (Campus Resource Booking):

- Actor: Admin → Add/Update Resource
- Actor: User → Create Booking, Cancel Booking, View Availability

5.3 Step 3: Create a use case diagram (quick guide)

The use case diagram is a **map** of your features:

- draw a system boundary rectangle (your application),
- put actors outside (roles),
- put use cases inside as verb phrases ("Create Booking", "Add Resource").

Tip: keep the diagram simple; use `include/extend` only if you truly need it.

5.4 Step 4: Write the use cases (template)

Write each use case in a structured way. Keep it short but complete:

- Use case name
- Primary actor
- Preconditions
- Trigger
- Main success scenario (numbered steps)
- Alternate flows (failures/edge cases)
- Postconditions
- Acceptance criteria (3–6 testable bullets)

5.5 Worked example: Use case "Create Booking" (full)

Use case name: Create Booking

Primary actor: User

Preconditions (examples):

- At least one resource exists in the system.
- Database connection is available.

Trigger: user clicks "Create Booking" / "Save Booking" button.

Main success scenario (example)

1. System shows booking form (resource, start time, end time).
2. User selects a resource and enters start/end time.
3. System validates input (mandatory fields, start $<$ end).
4. System checks overlap for the same resource.
5. System saves booking in DB.
6. System shows success message.
7. System updates booking list view (the new booking is visible).

Alternate flows (examples)

- **A1: Invalid time range** (start $\geq$ end)

  System shows validation error and does not save.

- **A2: Overlap exists**

  System shows "Overlapping booking" message and does not save.

- **A3: DB failure**

  System shows a friendly error message and does not crash.

Postconditions (examples)

- On success, booking record exists in DB and is visible in UI list.
- On failure, no record is created (data remains consistent).

Acceptance criteria (sample)

- Booking cannot overlap an existing booking for the same resource.
- Boundary case: end time equal to another booking's start time is allowed.
- Invalid input shows a clear message near the form and does not save.
- On success, booking is saved and visible in list view.

5.6 Step 5: Convert use cases into backlog items

One use case usually becomes **multiple issues** (UI + service + DAO + DB). **Example breakdown for "Create Booking":**

- UI: create booking form panel + input validation
- Service: `BookingService.createBooking(...)` with overlap rules
- DAO: `BookingDao.insert(...)` + query to find overlaps
- DB: schema constraints + indexes (by `resource_id`, time range)
- Test/demo: 3 scenarios (success, overlap, invalid time)

5.7 Step 6: Milestones (turn backlog into a plan)

Milestones help you avoid "everything at the end". Keep milestones aligned with Unit 06 deliverables:

- **Milestone-1 (Deliverable-1):** diagram + 3 written use cases
- **Milestone-2 (Deliverable-2):** class diagram + package structure
- **Milestone-3 (Deliverable-3):** DB schema + DAO CRUD for 1–2 entities
- **Milestone-4 (Deliverable-4):** UI integration + validations + end-to-end flow
- **Milestone-5 (final):** testing, packaging, polish, demo readiness

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

6 Deliverable-1: Use case diagram + 3 written use cases

Deliverable-1A: Use case diagram

Diagram should include:

- 2–3 actors,
- 6–10 use cases,
- relationships if needed (include/extend — optional).

Deliverable-1B: 3 written use cases (suggested)

Pick the most important core flows:

- Create Booking (core)
- Add Resource (admin)
- List/Search Bookings (read flow)

Minimum quality bar (what we will check)

- Each written use case has at least 6–10 steps in main success scenario.
- Each use case includes at least 2 alternate flows (invalid input + business rule failure).
- Acceptance criteria are testable and can be demonstrated during the final demo.

7 In-Class Activity (evidence)

- Add Deliverable-1 document to repo (PDF/Doc) or link in README.
- Create/update GitHub issues based on use cases (one issue per use case / feature).
- Add milestone labels or GitHub milestones (optional).

8 In-Class Exercises (with brief solutions)

1. Convert one Must-have feature into 2 requirements.

   Sample: "Create booking" → (1) booking must store resource + start + end, (2) booking must not overlap existing booking for same resource.

2. Identify 2–3 actors for your project.

   Sample: Admin, User/Student/Faculty.

3. Write 3 acceptance criteria for "Add Resource".

   Sample: name required; capacity positive; on success resource appears in list.

9 Self-Study Practice (no solutions)

1. Write a use case diagram for your project (6–10 use cases).
2. Write 2 complete written use cases using the template (include alternate flows).
3. Convert one use case into at least 5 GitHub issues (UI/service/DAO/DB/test).

10 Checkpoint Questions (with sample answers)

Q1. Why do we write acceptance criteria?

Sample answer: Acceptance criteria make requirements testable. They define what "done" means, reduce confusion, and help validate the system during demo.

Q2. What is the difference between an actor and a use case?

Sample answer: An actor is a role interacting with the system. A use case is a goal/action the actor performs using the system.

Q3. Why do we separate "main success scenario" and "alternate flows"?

Sample answer: It forces you to think about errors and edge cases early. Alternate flows often become validations and exception-handling code.